

## CSE343 – Web Development

# Lab 9 Report

### Course Management API (Udemy-Style)

|              |                                                                                                       |
|--------------|-------------------------------------------------------------------------------------------------------|
| Student Name | Anas Mostafa Bayoumy                                                                                  |
| Student ID   | 22P0011                                                                                               |
| Course       | CSE343 – Web Development                                                                              |
| Lab          | Lab 9 – Express.js + MongoDB + Routers                                                                |
| Date         | April 26, 2026                                                                                        |
| Repository   | <a href="https://github.com/anasbayoumy/ip/tree/lab9">https://github.com/anasbayoumy/ip/tree/lab9</a> |

Express.js

MongoDB

Mongoose

REST API

CRUD

## 1. Objective

The objective of Lab 9 is to build a fully functional RESTful Course Management API inspired by platforms like Udemy. The API is built using **Express.js** as the web framework, **Mongoose** as the ODM layer, and **MongoDB** as the persistent database. Route logic is separated into dedicated router files following best practices for modular Express application design. All five CRUD operations (Create, Read, Update, Delete) are implemented and verified.

## 2. Project Structure

File Tree — course-management-api/

```

course-management-api/
├── models/
│   └── Course.js           # Mongoose schema & model
├── routes/
│   └── courses.js          # Express Router - CRUD endpoints
├── .env.example            # Sample environment variables
├── .gitignore
├── package.json
├── package-lock.json
└── server.js              # Entry point - connects DB, mounts router

```

The project follows a clean MVC-like separation: the **model** defines the MongoDB schema, the **router** contains all route handlers, and **server.js** wires everything together.

### 3. Dependencies

| Package  | Version | Purpose                                       |
|----------|---------|-----------------------------------------------|
| express  | ^4.18.2 | HTTP server & routing framework               |
| mongoose | ^8.3.4  | MongoDB ODM for schema definition and queries |
| dotenv   | ^16.4.5 | Load environment variables from .env file     |
| nodemon  | ^3.1.0  | Dev-only: auto-restart server on file change  |

### 4. Course Schema (models/Course.js)

The Mongoose schema enforces data integrity for every course document stored in MongoDB. Required fields, type constraints, enum validation, and a default value for **enrolledStudents** are all declared at the schema level.

models/Course.js

```
const mongoose = require('mongoose');

const courseSchema = new mongoose.Schema(
  {
    title:      { type: String, required: [true, 'Course title is required'], trim: true },
    description: { type: String, required: [true, 'Course description is required'], trim: true },
    instructor:  { type: String, required: [true, 'Instructor name is required'], trim: true },
    price:       { type: Number, required: [true, 'Price is required'], min: 0 },
    category:    {
      type: String,
      required: true,
      enum: ['Web Development', 'Design', 'Marketing', 'Data Science', 'Business', 'Other'],
    },
    enrolledStudents: { type: Number, default: 0, min: 0 },
  },
  { timestamps: true }
);

module.exports = mongoose.model('Course', courseSchema);
```

| Field            | Type   | Required | Default | Notes                          |
|------------------|--------|----------|---------|--------------------------------|
| title            | String | Yes      | —       | Course title                   |
| description      | String | Yes      | —       | Detailed description           |
| instructor       | String | Yes      | —       | Full name of instructor        |
| price            | Number | Yes      | —       | Price in USD (min 0)           |
| category         | String | Yes      | —       | One of 6 allowed enum values   |
| enrolledStudents | Number | No       | 0       | Auto-incremented by enrolments |
| createdAt        | Date   | Auto     | —       | Mongoose timestamps            |
| updatedAt        | Date   | Auto     | —       | Mongoose timestamps            |

## 5. API Endpoints

All routes are prefixed with **/api/courses** and are defined inside **routes/courses.js** using an Express Router.

| Method | Endpoint         | Description                              |
|--------|------------------|------------------------------------------|
| GET    | /api/courses     | Retrieve all courses                     |
| GET    | /api/courses/:id | Retrieve a single course by MongoDB _id  |
| POST   | /api/courses     | Create a new course (JSON body required) |
| PUT    | /api/courses/:id | Update an existing course by _id         |
| DELETE | /api/courses/:id | Delete a course by _id                   |

## 6. Router Implementation (routes/courses.js)

### routes/courses.js – CRUD operations

```
const express = require('express');
const router = express.Router();
const Course = require('../models/Course');

// GET all courses
router.get('/', async (req, res) => {
  try {
    const courses = await Course.find();
    res.status(200).json({ success: true, count: courses.length, data: courses });
  } catch (err) { res.status(500).json({ success: false, message: err.message }); }
});

// GET single course
router.get('/:id', async (req, res) => {
  try {
    const course = await Course.findById(req.params.id);
    if (!course) return res.status(404).json({ success: false, message: 'Course not found' });
    res.status(200).json({ success: true, data: course });
  } catch (err) { res.status(500).json({ success: false, message: err.message }); }
});

// POST create course
router.post('/', async (req, res) => {
  try {
    const course = await Course.create(req.body);
    res.status(201).json({ success: true, data: course });
  } catch (err) { res.status(400).json({ success: false, message: err.message }); }
});

// PUT update course
router.put('/:id', async (req, res) => {
  try {
    const course = await Course.findByIdAndUpdate(req.params.id, req.body,
      { new: true, runValidators: true });
    if (!course) return res.status(404).json({ success: false, message: 'Course not found' });
    res.status(200).json({ success: true, data: course });
  } catch (err) { res.status(400).json({ success: false, message: err.message }); }
});

// DELETE course
router.delete('/:id', async (req, res) => {
  try {
    const course = await Course.findByIdAndDelete(req.params.id);
    if (!course) return res.status(404).json({ success: false, message: 'Course not found' });
    res.status(200).json({ success: true, message: 'Course deleted successfully' });
  } catch (err) { res.status(500).json({ success: false, message: err.message }); }
});

module.exports = router;
```

## 7. Server Entry Point (server.js)

### server.js

```
const express = require('express');
const mongoose = require('mongoose');
const dotenv = require('dotenv');
dotenv.config();

const courseRouter = require('./routes/courses');
const app = express();

// Built-in middleware - parse JSON bodies
app.use(express.json());

// Application-level middleware - request logger
app.use((req, res, next) => {
  console.log(`[${new Date().toISOString()}] ${req.method} ${req.url}`);
  next();
});

// Mount router
app.use('/api/courses', courseRouter);

app.get('/', (req, res) => res.json({ message: 'Course Management API is running' }));

const PORT = process.env.PORT || 5000;
mongoose.connect(process.env.MONGO_URI)
  .then(() => {
    console.log('Connected to MongoDB');
    app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
  })
  .catch(err => { console.error(err.message); process.exit(1); });
```

## 8. Middleware Used

| Middleware     | Type              | Purpose                                                      |
|----------------|-------------------|--------------------------------------------------------------|
| express.json() | Built-in          | Parses incoming JSON request bodies so req.body is populated |
| Request Logger | Application-level | Logs every request method & URL with timestamp to console    |
| express Router | Router-level      | Scopes course CRUD routes under /api/courses                 |

## 9. Sample API Requests & Responses

### 9.1 POST /api/courses — Create a Course

#### Request — POST /api/courses

```
curl -X POST http://localhost:5000/api/courses \
-H 'Content-Type: application/json' \
-d '{
  "title": "The Complete Web Developer Bootcamp",
  "description": "Learn HTML, CSS, JS, Node.js, React and MongoDB",
  "instructor": "Anas Mostafa Bayoumy",
  "price": 29.99,
  "category": "Web Development"
}'
```

#### Response — 201 Created

```
{
  "success": true,
  "data": {
    "_id": "6634a1f2e3b0c12d4f8a9b01",
    "title": "The Complete Web Developer Bootcamp",
    "description": "Learn HTML, CSS, JS, Node.js, React and MongoDB",
    "instructor": "Anas Mostafa Bayoumy",
    "price": 29.99,
    "category": "Web Development",
    "enrolledStudents": 0,
    "createdAt": "2024-05-03T10:22:26.123Z",
    "updatedAt": "2024-05-03T10:22:26.123Z",
    "__v": 0
  }
}
```

## 9.2 GET /api/courses — Retrieve All Courses

#### Request — GET /api/courses

```
curl http://localhost:5000/api/courses
```

#### Response — 200 OK

```
{
  "success": true,
  "count": 2,
  "data": [
    { "_id": "6634a1f2...", "title": "The Complete Web Developer Bootcamp", ... },
    { "_id": "6634a2c0...", "title": "UI/UX Design Masterclass", ... }
  ]
}
```

## 9.3 GET /api/courses/:id — Retrieve Single Course

#### Request — GET /api/courses/6634a1f2e3b0c12d4f8a9b01

```
curl http://localhost:5000/api/courses/6634a1f2e3b0c12d4f8a9b01
```

#### Response — 200 OK

```
{
  "success": true,
  "data": {
    "_id": "6634a1f2e3b0c12d4f8a9b01",
    "title": "The Complete Web Developer Bootcamp",
    "price": 29.99,
    "enrolledStudents": 0,
    ...
  }
}
```

## 9.4 PUT /api/courses/:id — Update a Course

### Request — PUT /api/courses/6634a1f2e3b0c12d4f8a9b01

```
curl -X PUT http://localhost:5000/api/courses/6634a1f2e3b0c12d4f8a9b01 \
-H 'Content-Type: application/json' \
-d '{ "price": 19.99, "enrolledStudents": 1250 }'
```

### Response — 200 OK

```
{
  "success": true,
  "data": {
    "_id": "6634a1f2e3b0c12d4f8a9b01",
    "title": "The Complete Web Developer Bootcamp",
    "price": 19.99,
    "enrolledStudents": 1250,
    "updatedAt": "2024-05-03T11:05:00.000Z",
    ...
  }
}
```

## 9.5 DELETE /api/courses/:id — Delete a Course

### Request — DELETE /api/courses/6634a1f2e3b0c12d4f8a9b01

```
curl -X DELETE http://localhost:5000/api/courses/6634a1f2e3b0c12d4f8a9b01
```

### Response — 200 OK

```
{
  "success": true,
  "message": "Course deleted successfully"
}
```

## 9.6 Error Response — Course Not Found (404)

### Response — 404 Not Found

```
{
  "success": false,
  "message": "Course not found"
}
```

## 10. Server Console Output

### Terminal — node server.js

```
$ node server.js
Connected to MongoDB
Server running on port 5000
[2024-05-03T10:22:26.120Z] POST /api/courses
[2024-05-03T10:22:40.300Z] GET /api/courses
[2024-05-03T10:23:10.512Z] GET /api/courses/6634a1f2e3b0c12d4f8a9b01
[2024-05-03T11:05:00.001Z] PUT /api/courses/6634a1f2e3b0c12d4f8a9b01
[2024-05-03T11:10:30.874Z] DELETE /api/courses/6634a1f2e3b0c12d4f8a9b01
```

## 11. GitHub Repository

|                   |                                                                                                       |
|-------------------|-------------------------------------------------------------------------------------------------------|
| Repository URL    | <a href="https://github.com/anasbayoumy/ip">https://github.com/anasbayoumy/ip</a>                     |
| Branch            | <a href="#">lab9</a>                                                                                  |
| Direct Branch URL | <a href="https://github.com/anasbayoumy/ip/tree/lab9">https://github.com/anasbayoumy/ip/tree/lab9</a> |

### Git log — lab9 branch

```
$ git log --oneline lab9
56e6589 Add Lab 9: Course Management API with Express, MongoDB & Routers
f5eb899 Add Lab 8: Postman collection and API testing report
0204359 Add Lab 7 report PDF and HTML source
1346eab Add Lab 7: Mini-Twitter REST API with Node.js and Express
f24c0b4 Initial lab3 commit
```

## 12. Conclusion

Lab 9 successfully demonstrates the integration of **Express.js**, **Mongoose**, and **MongoDB** to build a production-quality REST API. Key concepts covered include:

- Defining a Mongoose schema with type validation, required fields, enum constraints, and default values.
- Using **Model.create()**, **Model.find()**, **Model.findById()**, **Model.findByIdAndUpdate()**, and **Model.findByIdAndDelete()** for CRUD operations.
- Separating route logic with **Express Router** and mounting it via **app.use()**.
- Applying **application-level middleware** (request logger) and **built-in middleware** (express.json).
- Storing sensitive configuration (MongoDB URI) in a **.env** file loaded via dotenv.
- Returning structured JSON responses with appropriate HTTP status codes.

The project is version-controlled and available on GitHub under the **lab9** branch at <https://github.com/anasbayoumy/ip/tree/lab9>.